

Documents as Materialized Views:

Rising Above the Relational vs Document Database Debate

Stephen P. Tarzia, PhD, Sr. Staff Engineer,  MongoDB steve.tarzia@mongodb.com

Presented at the Greater Chicago Area Systems Research Workshop – May 11, 2026

Claim: The document model is a *performance tool* not a competing data modeling philosophy.

- Documents are a choice of physical materialization to create data locality.
 - Read queries are very fast if results come from just one doc instead of JOINing many rows.
- The relational model underlies all data, even if it's materialized as documents.

Side note: schema flexibility is an orthogonal concern:

- Flexibility is helpful during prototyping and app evolutions, but not in the steady state.

Physical design with documents:

- On relational DBs, “physical design” means just defining indexes, table partitioning, table layout, etc.
 - These affect workload performance, not semantics.
- On document DBs, you have more physical design choices:
 - How to materialize logical relations as physical documents: deciding when to embed arrays vs. doing joins (\$lookups), and when to duplicate data.
- Due to the expanded physical design space, document databases offer superior physical designs that are unavailable on relational databases.
 - But, on MongoDB today, achieving the ideal physical design is a challenging, manual, process!

Tradeoffs during physical design:

- Read vs. write:
 - More indexes and more denormalization (duplication) means more *write amplification*. That is, more write latency and more storage and cache overhead.
- Query1 vs query2:
 - For a fixed write budget, which queries should get their own ideally materialized documents.
 - Documents can be designed so that overlapping subsets of their content are useful for different queries
- Typical workloads have many 100s of queries!

What's missing from *today's* document DBs?

- No logical schemas:
 - Data manipulation languages work directly on physical materializations (documents).
 - At best, the system can guess the logical schemas.
 - Without logical schemas, all of the below improvements are impossible.
- No automatic, transactional maintenance of the document materializations:
 - If you choose to duplicate data, you have to manually keep it in sync on insert/update/delete.
 - Data updates should propagate to all materializations via the logical schema. This should be similar to index maintenance.
- No automatic workload-driven optimization of materializations.
 - The physical design space for document DBs is exponentially greater than for relational DBs.
 - Performance on a document DB can always be faster than on a relational DB, but this requires careful physical design.

Open Research Challenges:

1. Workload-driven physical schema (document) design optimization.
2. Efficient transactional maintenance of document materialized views.
3. Building an *adaptive materialization* layer on top of existing document DBs, like MongoDB.

MongoDB's C++ codebase is **open source**, and these research problems are wide open in the academic literature.

Related work:

- Couchbase, [Towards Principled, Practical Document Database Design](#)
 - Explained that schema design is important on doc DBs.
- Olteanu, [Factorized databases](#), SIGMOD Record 2016
 - Explained why nesting is space-efficient.
- Oracle, [JSON Relational Duality](#), SIGMOD'25
 - Presented JSON views, but not materialized!
- Stonebraker & Pavlo: [What goes around comes around... and around...](#) SIGMOD Record 2024:
 - NoSQL and relational systems are on a collision course.

Acknowledgements: Many thanks for help from David Storch, Anna Wawrzyniak, Ian Boros, Asya Kamsky, Thomas Chang, and Abhishek Chauhan.